



Assignment 2 — Mini Hackathon

Build for Sri Lanka · In-class group assessment · 4 hours

Assignment Title	Assignment 2 Mini Hackathon
Learning outcomes covered	LO1: Evaluate different types of software engineering frameworks used in web, mobile, cloud and AI-assisted software development. LO2: Apply frameworks to build software applications efficiently and effectively. LO3: Use best practices for integrating frameworks, managing collaborative development, applying CI/CD, ensuring code quality and deploying software solutions. LO4: Select appropriate frameworks, tools and agentic AI-assisted approaches to meet specific project and industry requirements.
Assignment Mode	Group — Assignment Group Conducted in class as a supervised four-hour build session. Every member must be present and must contribute.
Maximum Marks	100 marks Scored against the marking scheme in Section 3. Participation is recorded separately during the session.
Contribution to the Final Grade	15% of the final module mark (marked out of 100 and scaled). Together with the Assessment 1 (25%), this forms the 40% Assignments component of SE3090
Date published	4 th September 2026
Deadline for submissions	4 hour session Late submissions are not accepted; the hackathon is time-boxed by design.
Mode of Submission	A single PDF, renamed with your Group ID, uploaded through the course web (courseweb.sliit.lk). The PDF must contain the Git repository link, the deployed application link, the two-minute demonstration video link, team member names and IDs, a short problem/solution description, and the list of technologies and AI tools used.



1 Assignment Description

Modern engineering teams are small, cross-functional and AI-assisted, and they are judged on what runs rather than on what is promised. The Mini Hackathon compresses that reality into a single four-hour sprint, with one rule that matters: build something that helps a real person in Sri Lanka.

Working in officially registered teams of three or four, you will choose one genuine local problem, design a small but working solution, and deploy it to the public web — using any technology stack and any AI tools you like. Four hours is not a lot of time, so success comes from tight scoping, clear teamwork and pragmatic engineering.

1.1 The challenge

Theme — develop a practical digital solution for a current real-world problem in Sri Lanka. Each team identifies one existing problem that is affecting people, communities, businesses or the environment right now, then designs and builds a small but functional software solution that shows how technology can help.

Choose a problem you genuinely understand — depth beats breadth. A focused tool that solves one real pain point convincingly will always score better than an ambitious app that half-works.

***Suitable areas include** public transport, waste management, floods and disasters, education access, healthcare access, agriculture, tourism support, small-business digitalization, food waste, public-service information, energy and water saving, and community safety.*

Optional — Add an AI-Powered Feature

Teams may also include an AI-powered feature as an optional enhancement to make the solution more intelligent, useful, or innovative.

1.2 Rules and constraints

- **Teams:** Every registered member must contribute and be present for the session.
- The build happens during the four-hour session, between the announced start and end times.
- **Any technology is allowed** — any language, framework, database, API or platform. There are no stack restrictions.
- **AI tools are allowed and encouraged.** You own everything you submit: you must understand your code and be able to explain it in the demonstration.
- **You must deliver a functional solution.** A slide deck, a static mock-up or a non-working interface **will not be accepted**.
- Optional extras — an external API, authentication, maps or an AI feature — are welcome but not required given the time.
- All source code lives in your team's Git repository, with a meaningful commit history.
- The application must be deployed and publicly reachable.
- **Academic integrity:** AI assistance is fine; **submitting a pre-built or previous project as new work is not.**

1.3 Minimum software requirements

Your solution must include at least the following. Treat this as your *definition of done*.

1. A clear landing page or main user interface.
2. A short explanation of the selected Sri Lankan problem, shown inside the app.
3. At least two functional software features related to your solution.
4. At least one form that accepts user input.
5. Input validation with meaningful, friendly error messages.
6. A way to display, search, filter, calculate, update or process information.
7. A responsive interface that works on both desktop and mobile screens.
8. Basic navigation between the main sections or screens.
9. Sample data relevant to the chosen problem.
10. A clear demonstration of the solution's value to Sri Lankan users.

NOT ACCEPTED

A presentation-only concept, a static design, or an unfinished interface without working features will not be accepted. The project must actually run.

1.4 Team and roles

All registered team members share the work, and everyone writes code. As a starting point, divide ownership across these areas — roles can overlap, but each area should have a clear owner. In a three-member team, one member may combine two areas. All teams must meet the same core requirements; evaluators will consider the approved group size when judging scope, contribution and use of time. Record what each member did in the README.

Area	Typical focus
Problem & solution design	Frame the problem, define the MVP scope and the core user flow
UI development	Build the responsive interface, navigation and forms
Functional implementation	Build the features, data handling and validation logic
Testing, Git & deployment	Test and fix, manage the repository, deploy, and lead the demonstration

1.5 Deliverables

GIT REPOSITORY

Create a repository on GitHub, push the complete source, and keep a meaningful commit history. Your **README.md** must include: project title; the selected problem; the proposed solution; main features; technologies used; AI tools used; team member details and contributions; installation and execution instructions; the deployed application link; and the demonstration video link.

DEPLOYMENT

Deploy your app to a free hosting platform and make sure the public link works test it in a private or incognito window so you are not fooled by your own login or cache. Suitable free hosts include Vercel, Netlify, Render, Railway, Firebase Hosting and Cloud flare Pages. Pick one that matches your stack: static sites sit happily on Pages, Netlify or Vercel, while a Node or server backend is better on Render or Railway.

DEMONSTRATION VIDEO

Record a video of **no more than two minutes** covering your team and project briefly, the Sri Lankan problem you chose, your proposed solution, a demo of the main working features, the deployed application, and the expected impact. Upload it to OneDrive or any accessible platform and include the link.

1.6 Recommended four-hour schedule

Time	Phase	Focus
0–20 min	Plan	Select the problem and plan the solution
20–45 min	Design	Design the interface and divide responsibilities
45–175 min	Build	Implement the functional prototype
175–205 min	Polish	Test, fix issues and improve usability
205–225 min	Ship	Push the final code and deploy the application
225–240 min	Submit	Record the video and complete the submission

TIMING

Two timing rules save teams every year: lock your scope by minute 20, and stop building by minute 175 so you have time to deploy and record.

1.7 Final submission

Compile everything below into a single PDF, rename it with your **Group ID**, and submit it through the course web (courseweb.sliit.lk) before the end of the session, or as your lecturer instructs.

1. Git repository link
2. Deployed application link
3. Two-minute demonstration video link
4. Team member names and student IDs
5. A short description of the problem and your solution
6. The list of technologies and AI tools you used

2 Usage of AI

AI tools are permitted and encouraged for this assessment, in line with the CLEAR framework. Because the Mini Hackathon is a supervised, time-boxed, in-class build, what is being assessed is your ability to direct, evaluate and take ownership of AI-assisted work — not your ability to type every line unaided.

2.1 Where AI may be used

Task	Status
Researching the problem and framing the idea	Permitted
Generating UI layout, components and styling	Permitted
Generating, refactoring and debugging application code	Permitted
Producing realistic sample and test data	Permitted
Drafting the README and in-app problem description	Permitted
Deployment configuration and troubleshooting	Permitted
Writing the team contribution statement	Your own words
Submitting code no member can explain	Not permitted
Reusing a pre-built or previously submitted project	Not permitted

2.2 AI Prompt Log (mandatory)

Submit the same AI Prompt Log in the submission PDF. For each significant AI use, record the tool, exact prompt, purpose and how the output was checked or modified. Redact passwords, API keys and personal data.

2.3 Declaration (mandatory)

Every team must declare its AI usage in two places: the **README.md** in the repository, and the **submission PDF**. For each tool, state what it was used for in one line, for example: "*ChatGPT – generated the initial React form component; we rewrote the validation logic and added error messages.*" Teams that used no AI tools should state that explicitly. An undeclared AI dependency that becomes evident during the demonstration is treated as a breach of this declaration.

2.4 How this is verified

During the two-minute demonstration and the panel questions that follow, the evaluator may ask any member to explain a section of code, justify a design or stack decision, or make a small live modification. This is the primary check that the team understands and owns what it submitted.

3 Marking Scheme

The Mini Hackathon contributes 15% to your SE3090 grade. Rubric marks total 100 and are scaled to the evaluation component.

3.1 Mark allocation

Criterion	Marks	What strong work shows
Relevance of the Sri Lankan problem	10	A real, current, clearly explained local problem worth solving
Practicality & creativity of the solution	15	A sensible, original idea that plausibly helps real users
Minimum functional requirements	20	All ten requirements present and working
Quality & usability of the prototype	15	Clean, responsive and intuitive; handles bad input gracefully
Effective use of technology & AI tools	10	Good stack choices; AI used well and clearly understood
Git repository & documentation	10	Clear commit history; a complete, readable README
Successful deployment	10	A public link that works reliably for anyone
Quality of the 2-minute demonstration	5	Clear, on time, focused on the problem and the impact
Contribution from all registered members	5	Evidence that every registered member contributed meaningfully
Total	100	

3.2 Detailed marking rubric

Each criterion is marked in one of five bands. Award the band that best describes the evidence seen in the running application, the repository, the submission PDF and the demonstration. Where a team sits between two bands, award the lower band and note the reason on the mark sheet.

Criteria	Excellent	Good	Satisfactory	Poor	Very Poor
Relevance of the Sri Lankan problem (10 marks)	9–10 Real, current, specific local problem; clearly framed with the affected users named.	7–8 Genuine local problem, clearly explained; affected users described in general terms.	5–6 Recognisable problem; local relevance stated rather than demonstrated.	3–4 Vague or generic problem; only a weak link to Sri Lanka.	0–2 No identifiable problem, or no explanation given.
Practicality & creativity of the solution (15 marks)	13–15 Original, sensible idea; well scoped for four hours; plausibly helps real users.	11–12 Workable idea with some originality; scope mostly appropriate.	8–10 Conventional idea; the problem–solution link is implied, not argued.	5–7 Impractical or over-scoped; addresses the problem only loosely.	0–4 No coherent solution concept.
Minimum functional requirements (20 marks)	17–20 All ten requirements present and working reliably.	14–16 Eight or nine requirements working; minor gaps only.	10–13 Six or seven requirements working; the core flow is usable.	6–9 Three to five requirements working; the application only partly runs.	0–5 Two or fewer working, or the application does not run.
Quality & usability of the prototype (15 marks)	13–15 Clean, responsive and intuitive; validation handles bad input gracefully.	11–12 Usable and mostly responsive; validation present with minor rough edges.	8–10 Works, but layout or navigation is awkward; validation is basic.	5–7 Hard to use; breaks on common input; not responsive.	0–4 Interface unusable or incomplete.
Effective use of technology & AI tools (10 marks)	9–10 Stack choices justified; AI output reviewed and tested; code fully explained.	7–8 Sensible stack; AI used well; explanations mostly confident.	5–6 Stack works; AI output lightly reviewed; explanations patchy.	3–4 Poor tool fit; AI output used without understanding.	0–2 Cannot explain the submitted code; no ownership shown.
Git repository & documentation (10 marks)	9–10 Meaningful history from all registered members; README complete on all ten items.	7–8 Good commit history; README covers most required items.	5–6 Code pushed, but history thin or bulk-committed; README partial.	3–4 A single dump commit; README minimal.	0–2 No repository, or source code not pushed.
Successful deployment (10 marks)	9–10 Public link works reliably for anyone; deployed build matches the repository.	7–8 Deployed and reachable, with minor issues only.	5–6 Deployed but unstable or only partly functional.	3–4 Deployment attempted; link broken or runs locally only.	0–2 Not deployed.

**ASSIGNMENT 2**

Criteria	Excellent	Good	Satisfactory	Poor	Very Poor
Quality of the 2-minute demonstration (5 marks)	5 Within time; covers problem, solution, live features, deployment and impact.	4 Covers nearly all points; slightly over or under time.	3 Basics covered; unclear structure or one element missing.	2 Rushed or well over time; features barely shown.	0–1 No video, or the link is inaccessible.
Contribution from all registered members (5 marks)	5 Commits, README and demo evidence meaningful contributions from every registered member.	4 All registered members contributed; one member's share was noticeably smaller.	3 Uneven contribution; a smaller subset carried most of the work.	2 Only some members contributed; one or more were largely absent.	0–1 No evidence of shared contribution.
Total	100 marks.				